

ICS 2026 秋 Lab 0: Git 与 GitHub 实验报告

实验步骤

1. 确认当前分支与工作区状态。执行 `git branch` 查看分支列表，确认当前位于 `main` 分支（星号 * 标记）；执行 `git status` 确认工作区干净（working tree clean），避免后续切换分支时因存在未提交的修改而失败。
2. 在 `main` 分支修改 `main.c` 并提交。用 VSCode 打开 `main.c`，将 TODO 附近的输出语句修改为 `printf("this is from main");`，保存后依次执行 `git add main.c` 与 `git commit -m "second modify on main branch"`，生成提交 `a65f1f4`。
3. 创建并切换到 `feature` 分支。执行 `git switch -c feature`，从当前 `main` 分支创建新分支 `feature` 并完成切换；新分支会继承 `main` 分支已有的全部提交历史。
4. 在 `feature` 分支修改 `main.c` 的同一行并提交。再次用 VSCode 打开 `main.c`，将同一位置的输出语句修改为 `printf("this is from feature");`，保存后依次执行 `git add main.c` 与 `git commit -m "second modify on feature branch"`，生成提交 `b86e6a0`。至此，`main` 与 `feature` 两个分支对 `main.c` 的同一行持有了不同的修改内容，为合并冲突创造了条件。
5. 合并 `feature` 分支到 `main`，触发冲突。执行 `git switch main` 切回主分支，再执行 `git merge feature` 进行合并。终端输出 `Auto-merging main.c` 与 `CONFLICT (content): Merge conflict in main.c`，并提示 `Automatic merge failed; fix conflicts and then commit the result.`，表示合并冲突已出现（如图 1 所示）。

```

akane@akanedeMacBook-Air lab0-cnh % git add main.c
git commit -m "second modify on feature branch"

[feature b86e6a0] second modify on feature branch
1 file changed, 1 insertion(+)
akane@akanedeMacBook-Air lab0-cnh % git switch main
git merge feature

Switched to branch 'main'
Your branch is ahead of 'origin/main' by 2 commits.
(use "git push" to publish your local commits)
Auto-merging main.c
CONFLICT (content): Merge conflict in main.c
Automatic merge failed; fix conflicts and then commit the result.
akane@akanedeMacBook-Air lab0-cnh %

```

图 1 终端执行 git merge feature 时出现 CONFLICT 合并冲突报错

6. 查看冲突内容并手动解决。用 VSCode 打开 main.c, 可见 Git 在冲突位置插入了 <<<<<< HEAD、=====、>>>>>> feature 三行标记, 将 main 分支 (HEAD 侧) 与 feature 分支 (Incoming Change 侧) 的修改分隔开。手动保留需要的代码, 并删除全部冲突标记行后保存文件 (如图 2 所示)。

```

Users > akane > lab0-cnh > C main.c
1  #include <stdio.h>
2
3  int main()
4  {
5      // @TODO: print a sentence you want.
6      <<<<<< HEAD (Current Change)
7          //hello test collision
8          printf("this is from main");
9      =====
10         //hello test collision,and this is from feature
11         printf("this is from feature");
12     >>>>>> feature (Incoming Change)
13         printf("Hello, world!\n");
14         printf("你好你好呀, 这是陈南晖的lab0作业");
15     }

```

图 2 VSCode 中 main.c 的合并冲突标记及两侧代码

7. 完成合并提交。执行 `git add main.c` 将解决后的文件重新加入暂存区，再执行 `git commit -m "resolve merge conflict"` 生成合并提交 `f03beff`，合并冲突正式解决。

8. 验证提交历史。执行 `git log --oneline` 查看提交记录，最新一条为 `resolve merge conflict (f03beff)`，其下依次为 `main` 分支与 `feature` 分支各自的修改提交，证明分支合并与冲突解决均已完成（如图 3 所示）。

```
[main f03beff] resolve merge conflict
akane@akanedeMacBook-Air lab0-cnh % git log --oneline

f03beff (HEAD -> main) resolve merge conflict
b86e6a0 (feature) second modify on feature branch
a65f1f4 second modify on main branch
c1eed82 modify main.c on feature branch
4e929be modify main.c on main branch
bef7c99 (origin/main, origin/HEAD) finish main.c TODO
1089691 Initial commit
```

图 3 `git log --oneline` 显示的提交历史

补充说明：本次实验中，第一次尝试合并时两个分支对 `main.c` 的修改未落在同一位置，合并以 `Fast-forward`（快进合并）方式直接完成、未产生冲突。按照实验文档的提示，无需回撤已有提交，继续在 `main` 分支追加对同一行的修改并提交后再次合并，最终成功触发冲突并完成解决。该过程也验证了「冲突产生于两个分支对同一文件同一位置的不同修改」这一原理。

起始三问：

1. 有过简单的多人协同开发经历。当时团队成员各自负责项目的不同功能模块，主要通过微信互传代码文件，以复制粘贴的方式合并彼此的代码来完成协作。

2. 通过暂存区可以区分尚未整理完毕的代码，和已经审核完毕、逻辑完整、适合生成版本快照的成熟代码。我们可以从众多修改中挑选一部分改动加入暂存区，再统一提交，做到精准控制每一次提交包含的内容，让版本历史更加清晰。

3. git branch: 只列出**本地分支**，仅展示存放在本机仓库的分支；

git branch -a: -a 是 all 的缩写，会同时列出**本地分支**和远程仓库的所有远程跟踪分支，可以看到远端存在但本机尚未创建的分支。

文章阅读感悟

1. 《Commit message 和 Change log 编写指南》

文章讲解 Git 的提交信息编写规范，推荐 Angular 规范，将提交信息划分为头部、正文、脚注，区分 feat、fix、docs 等改动类型。规范的提交说明便于查阅历史、筛选改动，还可借助工具自动生成项目更新日志。

2. 《Gitflow 使用规范》

本文介绍 Gitflow 经典分支协作模式。设置 main、develop 长期分支，搭配 feature、release、hotfix 临时分支，规定分支之间的合并流程。通过分支隔离不同工作，实现功能开发、版本发布、线上故障修复互不干扰，适合团队正式项目开发。

对“为什么要学习 Git”的理解

没有 Git 时，代码依靠复制、聊天软件传输，版本混乱，出错难以回退。

- 1.版本回溯：每次提交保存代码快照，出错可以回退，不用手动备份大量旧文件。
- 2.多人协作：利用分支并行开发，合理处理代码冲突，避免直接复制粘贴带来的覆盖问题。

3.便于维护：规范的提交记录能清晰体现改动目的，方便自己和团队阅读项目历史。

4.备份与分享：搭配 GitHub 远程托管代码，防止本地文件丢失，也便于代码共享。